CSE 598 008
EECS 498 003
Lab 11

Tianyiming (Henry) Jing

November 13, 2026

Reminder

- Problem Set 4 due November 23
- Project 2 released November 24
  - Groups of up to two: start looking for a teammate

Today

- Refinement exercise: a distributed multiset
- Questions about PS4? Ask anytime

A Distributed Multiset

```
datatype Event =
    | Insert(item: int)
    | Remove(item: int)
    | NoOp

// The spec: one multiset,
// initially empty
datatype Variables =
    Variables(items: multiset<int>)
```

- A multiset is like a set, but an item can appear more than once
- `Remove(item)` can happen only when `item` is in the multiset
- The multiset is spread over several hosts; each host keeps its own multiset
- A host can send one of its items to another host: the item waits in that host's inbox until it is received

Host Steps

```
LocalInsert(item)
LocalRemove(item)
SendItem(item, dest)
ReceiveItem

datatype Message =
    Transfer(item: int, dest: HostId)
```

- The network keeps one inbox per host, and each inbox is a multiset of items
- Sending `Transfer(item, dest)` adds `item` to the inbox of `dest`; receiving it takes `item` out
- Which event does each step produce? You decide

Multisets in Dafny

```
var m: multiset<int> := multiset{1, 1, 2};

assert m[1] == 2;      // number of copies of 1
assert 1 in m && 3 !in m;

var m2 := m + multiset{3};   // add one copy of 3
var m3 := m - multiset{1};   // remove one copy of 1

assert m3 == multiset{1, 2};
assert |m2| == 4;      // size, counting copies
```

Refinement Proof (from Lecture)

```
lemma RefinementInit(v)
    requires Init(v)
    ensures Inv(v)
    ensures Spec.Init(Abstraction(v))

lemma RefinementNext(v, v', evt)
    requires Next(v, v', evt)
    requires Inv(v)
    ensures Inv(v')
    ensures Spec.Next(Abstraction(v),
                      Abstraction(v'), evt)
```

- Abstraction: the spec state that a state of the system stands for
- Each step of the system is a spec step with the same event
- A stutter step is a spec `NoOp` step: the spec state does not change
- The abstraction function is untrusted: the events keep it honest (Thursday's lecture)

Exercise

- Starter code: `lab11.dfy` on the course GitHub
  - `UtilitiesLibrary.dfy` is in the same folder
- Part 1: fill in the four steps of the host
  - The signatures are given; each comment says what the step does
  - Decide which event each step produces
- Part 2: prove that the system refines the spec
  - Write the abstraction function and an invariant
  - Prove `RefinementInit` and `RefinementNext`
  - Define additional functions and lemmas as needed

Extra - The Blue-Eyed Islanders

- An island of perfect logicians: 100 have blue eyes, 100 have brown eyes
  - Everyone sees everyone else's eyes, but never their own (no mirrors, and nobody talks about it)
  - Anyone who figures out they have blue eyes must leave on the ferry that night
- One day, a visitor tells everyone: "At least one of you has blue eyes."
- What happens?
  - Think first: what if only one islander had blue eyes? What if two did?

Extra - The Blue-Eyed Islanders

- One blue-eyed islander: sees no blue eyes, so it must be them; leaves on night 1
- Two: each sees one; when that one does not leave on night 1, each realizes there is a second, and both leave on night 2
- By induction: with n blue-eyed islanders, all of them leave on night n
  - So all 100 leave on night 100
- But everyone already knew “at least one has blue eyes”. What did the visitor add?
  - Common knowledge: everyone knows that everyone knows that … everyone knows it

https://en.wikipedia.org/wiki/Common_knowledge_(logic)